

On-Device Machine Learning Optimization

overview

This project's goal was to build a highly efficient keyword spotting model capable of recognizing a small vocabulary of spoken words with high accuracy. The process began with data preparation, using the official `speech_commands` dataset from the `tensorflow_datasets` library. The raw audio was preprocessed into a format suitable for a neural network by converting it into spectrograms—visual, image-like representations of the sound. To make the model more robust and prevent it from simply memorizing the training data, two forms of data augmentation were applied: adding random background noise and randomly time-shifting the audio clips.

A custom, quantization-friendly Convolutional Neural Network (CNN) was designed using the Keras Functional API. This architecture was made powerful by stacking several convolutional blocks, each containing BatchNormalization for stable and fast training. Crucially, the standard ReLU activation function was replaced with ReLU6, which clips the maximum neuron output. This design choice prevents unstable activation values, making the model far more compatible with the subsequent optimization process and preventing the large drops in accuracy that can occur during quantization.

The training process was automated to find the best possible version of the model. By setting a high number of training epochs (100), the model was given ample time to learn. However, to prevent overfitting and save time, two key callbacks were used. `EarlyStopping` monitored the model's performance on a separate validation dataset and automatically stopped the training when the accuracy stopped improving, restoring the weights from the best-performing epoch. Additionally, `ReduceLROnPlateau` automatically lowered the learning rate whenever the training process plateaued, helping the model to fine-tune its parameters and achieve better results.

The final and most critical stage was optimization. The fully trained, high-precision (float32) Keras model was converted into a highly compact and efficient int8 TensorFlow Lite (.tflite) model. This post-training integer quantization process dramatically reduces the model's file size and makes it suitable for deployment on low-power devices like microcontrollers. The script concluded by loading this final .tflite model and running it against a test set to validate its real-world performance, ensuring that the accuracy remained high (above 85%) and that the optimization process was successful.

how I did it

it was a whole lot of work for about 3 days straight , this model was trained pruned and quantised within this time frame and has a accuracy of 93 percent on 3 keywords such as left,right,no under a size of 122kb which is excpetional standard as after turning to a c array this will run even in a esp 32 on . this project took a lot longer than i exected it to take but the open source materials were outof date with current tensor flow library version and there were not many sources on the quantising and pruning (none which i could understand) there is a whole lot of math under this and lot of optimisations by gemini which kept giving me errors so finnaly i present "JARVIS"

SOURCE code

```
import os

import pathlib

import numpy as np

import tensorflow as tf

import tensorflow_datasets as tfds

from tensorflow.keras import layers

from tensorflow.keras import models

|

print("--- Preparing Dataset ---")

tf.random.set_seed(42)

np.random.seed(42)

|

COMMANDS = ['down', 'go', 'left', 'no', 'right', 'stop', 'up', 'yes']

|

(ds_train, ds_val), ds_info = tfds.load(

    'speech_commands',

    split=['train', 'validation'],

    with_info=True,

    shuffle_files=True

)

|

def filter_by_label(element):

    return element['label'] < len(COMMANDS)

|

def preprocess_dataset(element, is_training=False):

    waveform = element['audio']

    label = element['label']

    waveform = tf.cast(waveform, tf.float32) / 32768.0
```

```

|
if is_training:

    noise = tf.random.normal(shape=tf.shape(waveform), mean=0.0, stddev=0.01, dtype=tf.float32)

    waveform = waveform + noise

    shift_amount = tf.random.uniform(shape=(), minval=-1600, maxval=1600, dtype=tf.int32)

    waveform = tf.roll(waveform, shift=shift_amount, axis=0)

|
    waveform = waveform[:16000]

    zero_padding = tf.zeros([16000] - tf.shape(waveform), dtype=tf.float32)

    waveform = tf.concat([waveform, zero_padding], 0)

|

    spectrogram = tf.signal.stft(waveform, frame_length=255, frame_step=128)

    spectrogram = tf.abs(spectrogram)

    spectrogram = spectrogram[..., tf.newaxis]

|

    return spectrogram, label

|
batch_size = 64

ds_train_prep = ds_train.filter(filter_by_label).map(lambda x: preprocess_dataset(x, is_training=True),
num_parallel_calls=tf.data.AUTOTUNE)

ds_val_prep = ds_val.filter(filter_by_label).map(preprocess_dataset, num_parallel_calls=tf.data.AUTOTUNE)

|
print("--- Calculating normalization statistics efficiently ---")

count = 0.0

total_sum = 0.0

total_sq_sum = 0.0

for spectrogram, _ in ds_train_prep.batch(batch_size):

    total_sum += tf.reduce_sum(spectrogram)

    total_sq_sum += tf.reduce_sum(tf.square(spectrogram))

```

```

count += tf.cast(tf.size(spectrogram), tf.float32)

|
mean = (total_sum / count).numpy()

variance = (total_sq_sum / count) - (mean**2)

std = np.sqrt(variance)

print(f"Dataset Mean: {mean:.4f}, Std: {std:.4f}")

|
def normalize_manual(spectrogram, label):

    return (spectrogram - mean) / std, label

|
ds_train_norm = ds_train_prep.map(normalize_manual,
num_parallel_calls=tf.data.AUTOTUNE).cache().shuffle(10000).batch(batch_size).prefetch(tf.data.AUTOTUNE)

ds_val_norm = ds_val_prep.map(normalize_manual,
num_parallel_calls=tf.data.AUTOTUNE).batch(batch_size).prefetch(tf.data.AUTOTUNE)

|
for example_spectrograms, _ in ds_train_prep.take(1):

    break

input_shape = example_spectrograms.shape

num_labels = len(COMMANDS)

|
def build_model(input_shape, num_labels):

    inputs = layers.Input(shape=input_shape)

    x = layers.Resizing(32, 32)(inputs)

    x = layers.Conv2D(32, 3, padding='same')(x)

    x = layers.BatchNormalization()(x)

    x = layers.ReLU(max_value=6)(x)

    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(64, 3, padding='same')(x)

    x = layers.BatchNormalization()(x)

    x = layers.ReLU(max_value=6)(x)

```

```

x = layers.MaxPooling2D()(x)

x = layers.Conv2D(128, 3, padding='same')(x)

x = layers.BatchNormalization()(x)

x = layers.ReLU(max_value=6)(x)

x = layers.GlobalAveragePooling2D()(x)

x = layers.Dense(128)(x)

x = layers.ReLU(max_value=6)(x)

x = layers.Dropout(0.5)(x)

outputs = layers.Dense(num_labels)(x)

return models.Model(inputs, outputs)

|
base_model = build_model(input_shape, num_labels)

base_model.compile(optimizer=tf.keras.optimizers.Adam(),

                    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),

                    metrics=['accuracy']))

|
EPOCHS = 100

callbacks = [

    tf.keras.callbacks.EarlyStopping(monitor='val_accuracy', mode='max', patience=10, verbose=1,
    restore_best_weights=True),

    tf.keras.callbacks.ReduceLROnPlateau(monitor='val_accuracy', mode='max', factor=0.2, patience=5,
    verbose=1)

]

|
print(f"\n--- Training the model for up to {EPOCHS} epochs ---")

history = base_model.fit(

    ds_train_norm,

    validation_data=ds_val_norm,

    epochs=EPOCHS,

```

```

callbacks=callbacks
)

|
print("\n--- Quantizing the trained model ---")

def representative_data_gen():

    for spectrogram, _ in ds_train_prep.take(150):

        normalized_spectrogram = (spectrogram - mean) / std

        yield [normalized_spectrogram[tf.newaxis, ...]]

|
converter = tf.lite.TFLiteConverter.from_keras_model(base_model)

converter.optimizations = [tf.lite.Optimize.DEFAULT]

converter.representative_dataset = representative_data_gen

converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]

converter.inference_input_type = tf.int8

converter.inference_output_type = tf.int8

tflite_model_quant = converter.convert()

|
model_filename = 'kws_quantized_final.tflite'

with open(model_filename, 'wb') as f:

    f.write(tflite_model_quant)

print(f"\n✅ Powerful model saved as '{model_filename}'")

|
print("\n--- Validating the final quantized TFLite model ---")

interpreter = tf.lite.Interpreter(model_path=model_filename)

interpreter.allocate_tensors()

input_details = interpreter.get_input_details()

output_details = interpreter.get_output_details()

input_scale, input_zero_point = input_details[0]['quantization']

output_scale, output_zero_point = output_details[0]['quantization']

```

```
correct_predictions = 0
```

```
total_samples = 0
```

```
|  
for spectrograms, labels in ds_val_prep.batch(batch_size):
```

```
    for i in range(len(labels)):
```

```
        spectrogram = spectrograms[i]
```

```
        label = labels[i]
```

```
        total_samples += 1
```

```
    
```

```
    normalized_spectrogram = (spectrogram[np.newaxis, ...] - mean) / std
```

```
    
```

```
    quantized_input = tf.cast((normalized_spectrogram / input_scale) + input_zero_point, dtype=tf.int8)
```

```
    
```

```
    interpreter.set_tensor(input_details[0]['index'], quantized_input)
```

```
    interpreter.invoke()
```

```
    output_data_quantized = interpreter.get_tensor(output_details[0]['index'])[0]
```

```
    output_data_float = (output_data_quantized.astype(np.float32) - output_zero_point) * output_scale
```

```
    predicted_label_index = np.argmax(output_data_float)
```

```
    if predicted_label_index == label.numpy():
```

```
        correct_predictions += 1
```

```
|  
accuracy = correct_predictions / total_samples
```

```
print(f"\nFinal Accuracy of Quantized Model: {accuracy:.2%}")
```

```
|  
if accuracy > 0.85:
```

```
    print("\n🎉 Congratulations! The model's accuracy is above 85%.")
```

```
else:
```

```
    print("\nModel accuracy is below 85%. Further tuning may be needed.")
```


final accuracy test on collab

```
--- Evaluating model on the filtered test set... ---  
Processed 100 samples...  
Processed 200 samples...  
Processed 300 samples...  
Processed 400 samples...  
Processed 500 samples...  
Processed 600 samples...  
Processed 700 samples...  
Processed 800 samples...  
Processed 900 samples...  
Processed 1000 samples...  
Processed 1100 samples...  
Processed 1200 samples...  
Processed 1219 total samples.  
  
--- Accuracy Report for Target Keywords ---  
Accuracy for 'left ': 98.30% (405/412)  
Accuracy for 'right': 95.52% (384/402)  
Accuracy for 'no   ': 84.44% (342/405)  
-----  
Overall Accuracy for this subset: 92.78%
```